

ToSh Cheat Sheet: Control Flow And Matching

Branching, loops, errors, cleanup, and expression-style matching

parser/tests first

If And Conditional Style

```
if ($age >= 18) {
  echo adult
} else if ($age >= 13) {
  echo teen
} else {
  echo child
}
```

```
var label = if ($count == 1) { "item" } else { "items" }
echo $"You have { $count } { $label }."
```

if works both as a statement and as an expression, which makes it handy inside assignments and interpolation.

Loops

```
for item in (ls | first 3) {
  echo $item.Name
}
```

```
for i in 1..5 {
  echo $i
}
```

```
while (($count < 3)) {
  echo $count
  $count += 1
}
```

```
until (($ready)) {
  sleep 1s
}
```

Keyword	Use
break	Exit the current loop.
continue	Skip to the next iteration.
return	Exit the current function/script.
throw	Raise an error value.

Errors And Cleanup

```
try {
  read-file config.json | from json
} catch (err) {
  writeline $"Error: { $err.Message }"
} finally {
  writeline done
}
```

```
func process-file(path) {
  var handle = (open-file --read $path)
  defer { close $handle }

  return (read-to-end $handle)
}
```

- catch (err) binds the thrown value or exception object.
- finally always runs.
- defer schedules cleanup when the current scope exits.

Switch

```
switch ($state) {
  case file { echo file }
  case dir { echo dir }
  default { echo other }
}
```

```
switch ($temperature) {
  case < 32 { echo Freezing }
  case 32..50 { echo Cold }
  case > 75 { echo Hot }
}
```

switch is statement-style and does not fall through. Each case is independent; no extra break is needed.

Match Expressions

```
var label = match ($status) {
  200 => "OK"
  404 => "Not Found"
  default => "Other"
}
```

```
match ($value) {
  _ is int => "integer"
  100..199 => "informational"
  _ =~ "\\..csv$" => "csv"
  default => "fallback"
}
```

Guards And Ordering

```
match ($n) {
  0 => "zero"
  _ > 0 if (($n % 2 == 0)) => "positive even"
  _ > 0 => "positive odd"
  default => "other"
}
```

- Arms are tested top to bottom.
- The first matching arm wins.
- default is required for exhaustive behavior.
- Guards use if (...) after the pattern.

Control Patterns In Scripts

```
func recent(span: TimeSpan) =>
  ls -la | where _.Modified > ((date now) - $span)
```

```
func big-files(min: StorageSize) {
  ls -la | where _.Type == file | where _.Size >= $min
}
```

```
echo $"2 + 2 = {2 + 2}"
echo $"Files: {ls | where _.Type == file | count}"
echo ((date now) - 2d)
```

Helpful Mental Model

- Use if when you want branch-oriented control flow.
- Use match when you want a value back from a compact pattern table.
- Use switch when each arm performs statements or side effects.
- Pair loops with pipeline commands whenever you can; fall back to loops when stateful control is clearer.

ToSh scripts stay readable when most data reshaping happens in pipelines, while loops and match handle the control decisions around those pipelines.

Pattern Catalog

```
200 => "OK"           # exact value
_ is int => "integer"  # runtime type
_ >= 90 => "A"         # comparison
100..199 => "info"     # range
_ =~ "\\\.csv$" => "csv" # regex
_ > 0 if (($n % 2 == 0)) => "even"
```

- Match arms are checked top to bottom.
- The first matching arm wins.
- Keep **default** last and obvious.

Match Recipes

```
func http_category(status: int) -> string {
  return match ($status) {
    100..199 => "Informational"
    200..299 => "Success"
    300..399 => "Redirection"
    400..499 => "Client Error"
    500..599 => "Server Error"
    default => "Unknown"
  }
}

match ($file) {
  _ =~ "\\\.pdf$" => "document"
  _ =~ "\\\. (csv|tsv)$" => "data"
  _ =~ "\\\.tar\." => "archive"
  default => "other"
}
```

Switch Recipes

```
switch ($state) {
  case Inv.StockState.Low { echo low }
  case Inv.StockState.Out { echo out }
  default { echo ok }
}
```

Try, Throw, Return

```
try {
  throw "boom"
} catch (err) {
  echo $err
} finally {
  writeline done
}

func find-first(items, predicate) {
  for item in $items {
    if (invoke $predicate $item) {
      return $item
    }
  }
  return null
}
```

- **return** can bubble out from nested loops and blocks.
- Unhandled **throw** becomes a top-level shell diagnostic.
- **catch (err)** gives you the thrown value or exception object.

Defer Patterns

```
func process-file(path) {
  var handle = (open-file --read $path)
  defer { close $handle }

  return (read-to-end $handle)
}

defer { echo "third" }
defer { echo "second" }
defer { echo "first" }
```

defer runs in LIFO order. It is the easiest way to clean up file handles, temp resources, or partial state when a scope exits early.

Loop Recipes

```
for proc in (ps | sort Memory | reverse | first 3) {
  writeline $proc.Name
}

echo one skip two three | each {
  if ((_ == "skip")) { continue }
  echo _
}
```

Real Script Patterns

```
ls -la | first 10 | each {
  var tag = match (_,_.Type.ToString()) {
    "dir" => "DIR "
    "file" if ((_Size > 1kb)) => "BIG "
    "file" => "file"
    "symlink" => "link"
    default => "???"
  }
  echo $"[$tag] {_.Name}"
}

func classify_score(score: int) -> string {
  return match ($score) {
    _ >= 90 => "A"
    _ >= 80 => "B"
    _ >= 70 => "C"
    _ >= 60 => "D"
    default => "F"
  }
}
```

Choose The Form

Use	When it fits best
if	one-off branching with side effects or short expressions
match	compact value selection from ordered patterns
switch	statement blocks keyed from one tested value
loops	stateful iteration when pipelines are not enough
try/defer	failure handling and cleanup

A readable ToSh script usually keeps data reshaping in pipelines, then uses **match**, **if**, and **loops** only where control flow really matters.